Clase 1 - Fundamentos de la ingeniería de software

Relación entre Ing de SW, de Datos y de IA

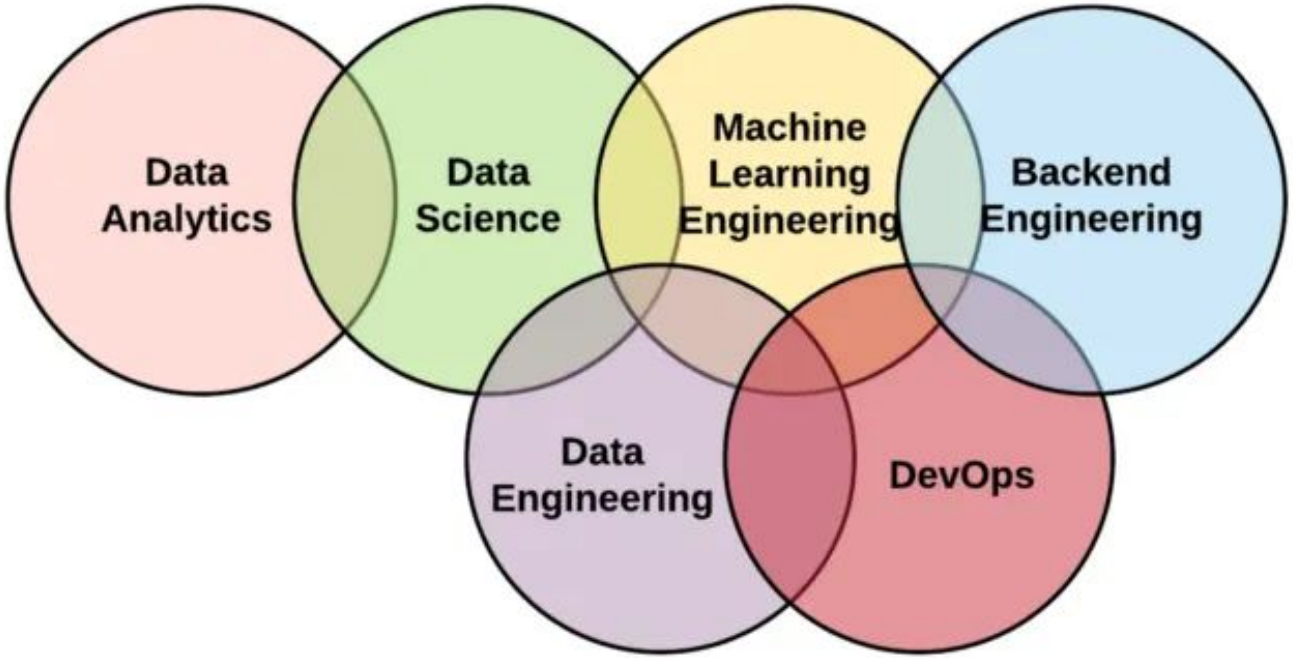
Cómo encaja la Ing de SW en el perfil del Ingeniero de IA

- Base estructural: Un sistema productivo de IA es software lo requiere especificación, diseño, pruebas y mantenimiento del sistema completo.
- Gestión de complejidad: La Ing. de SW aporta metodologías para gestionar software, datos y sistemas distribuidos utilizados en IA.
- Calidad y confiabilidad: Asegura que los sistemas de IA sean seguros, eficientes, mantenibles y éticamente responsables.
- Integración en sistemas reales: La IA se despliega dentro de aplicaciones, APIs y entornos críticos (salud, finanzas, transporte) sujetos a variedad de restricciones.

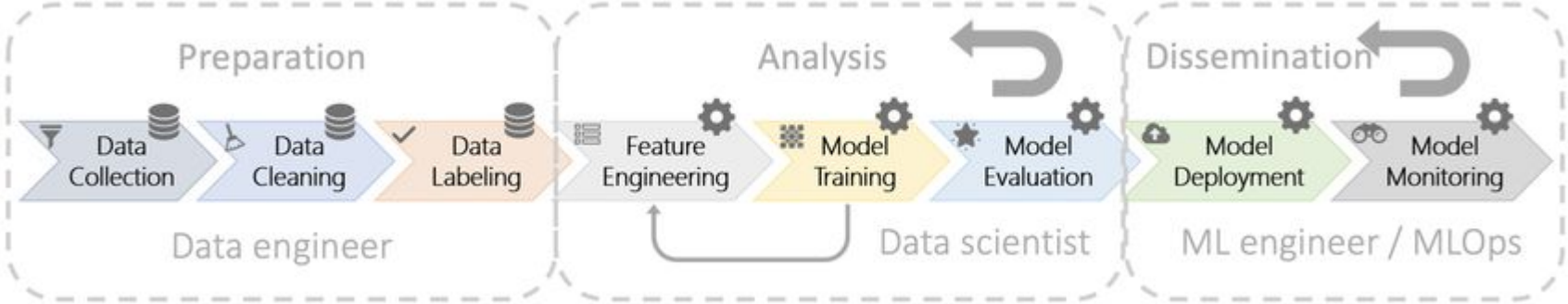
Cómo encaja la Ing Datos en el perfil del Ingeniero de IA

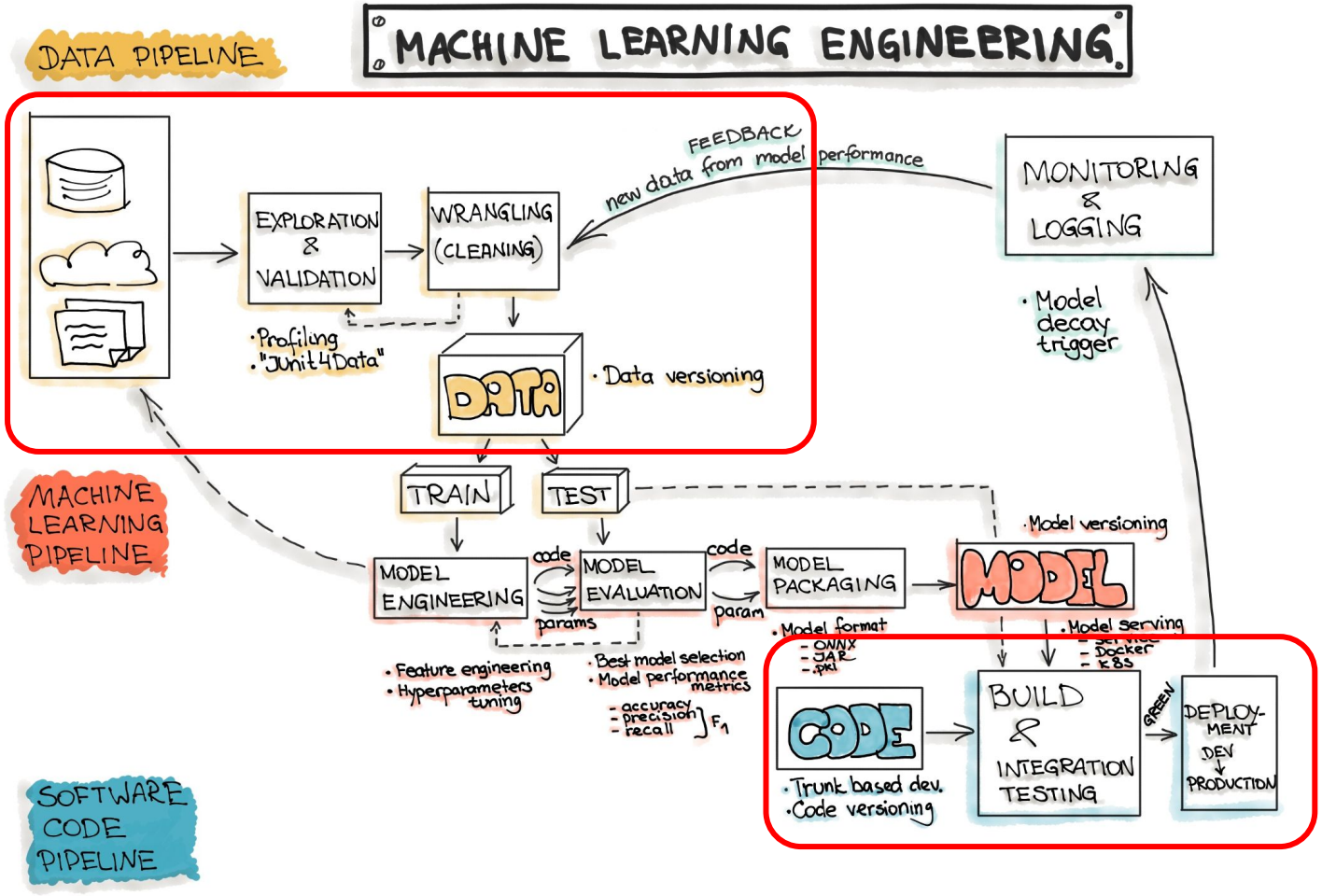
- Infraestructura y pipelines: Construye y mantiene arquitecturas de datos (ETL/ELT, streaming, lagos de datos) que permiten alimentar y escalar modelos de IA.
- Gobernanza y calidad: Define políticas de seguridad, privacidad, linaje y control de versiones de datos, fundamentales en aplicaciones sensibles como salud y finanzas.
- Optimización y eficiencia: Garantiza disponibilidad, volumen, velocidad y veracidad de datos, optimizando costos de almacenamiento y tiempos de procesamiento.

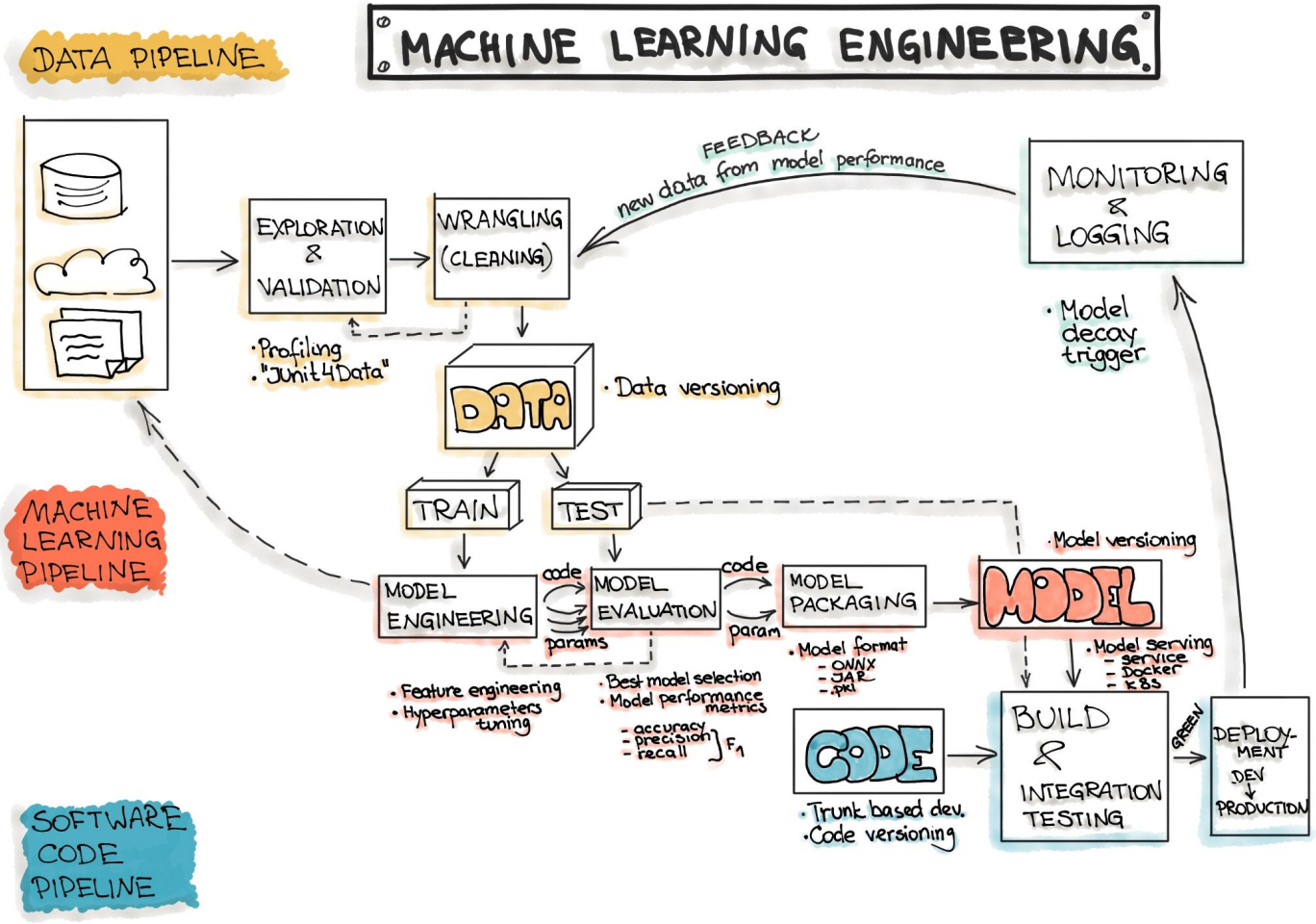
Diferentes Roles



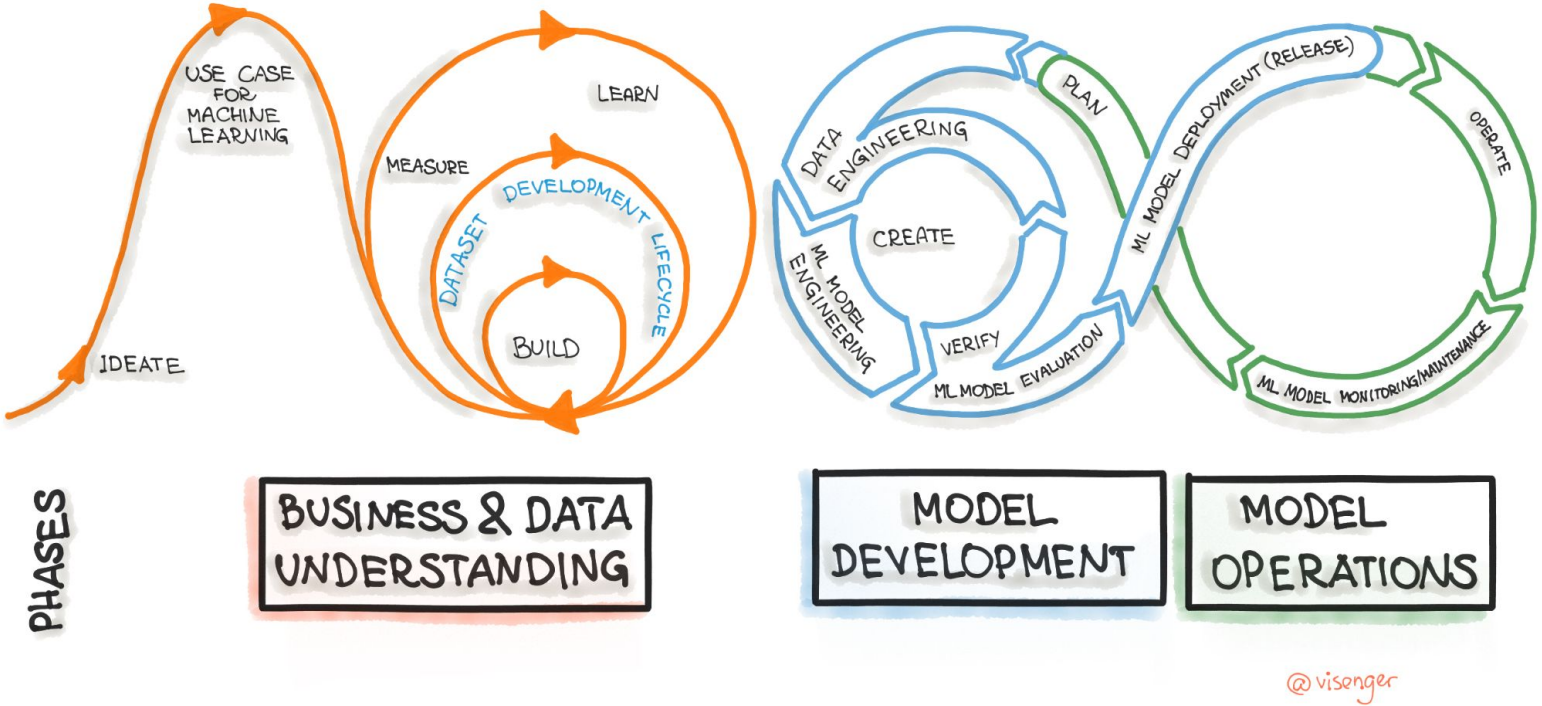
Diferentes Roles




DATA PIPELINE
MACHINE LEARNING ENGINEERING
EXPLORATION & VALIDATION
WRANGLING (CLEANING)
FEEDBACK
new data from model performance
MONITORING & LOGGING
· Profiling
· "Junit4Data"
DATA
· Data versioning
· Model decay trigger
MACHINE LEARNING PIPELINE
TRAIN
TEST
· Model versioning
MODEL ENGINEERING
code
params
MODEL EVALUATION
code
param
MODEL PACKAGING
MODEL
· Model format
- ONNX
- JAR
- .pkl
· Model serving
- Docker
- K8s
· Feature engineering
· Hyperparameters tuning
· Best model selection
· Model performance metrics
- accuracy
- precision
- recall
F1
CODE
· Trunk based dev.
· Code versioning
BUILD & INTEGRATION TESTING
GREEN
DEPLOY-MENT
DEV
PRODUCTION
SOFTWARE CODE PIPELINE


DATA PIPELINE
MACHINE LEARNING ENGINEERING
EXPLORATION & VALIDATION
WRANGLING (CLEANING)
FEEDBACK
new data from model performance
MONITORING & LOGGING
· Profiling
· "Junit4Data"
DATA
· Data versioning
· Model decay trigger
MACHINE LEARNING PIPELINE
TRAIN
TEST
· Model versioning
MODEL ENGINEERING
code
params
MODEL EVALUATION
code
param
MODEL PACKAGING
MODEL
· Model format
- ONNX
- JAR
- .pkl
· Model serving
- service
- Docker
- K8s
· Feature engineering
· Hyperparameters tuning
· Best model selection
· Model performance metrics
- accuracy
- precision
- recall
F1
CODE
· Trunk based dev.
· Code versioning
BUILD & INTEGRATION TESTING
GREEN
DEPLOY-MENT
DEV
PRODUCTION
SOFTWARE CODE PIPELINE

Metodología CRISP-ML(Q)



Características Propias del Sw

Característica Esenciales

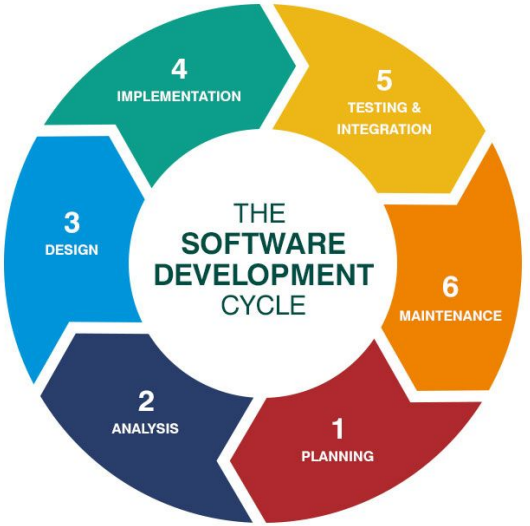
- **Aceptabilidad**: El software debe ser comprensible, usable y compatible con sistemas ya utilizados.
- **Confiabilidad y seguridad**: Debe ser confiable, seguro y evitar daños físicos o económicos.
- **Eficiencia**: No debe desperdiciar recursos; incluye tiempos de respuesta y uso de memoria.
- **Mantenibilidad**: Debe poder evolucionar fácilmente para satisfacer necesidades cambiantes de clientes.

Complejidades del Software

- **Esencial**: Proviene del dominio del problema
  - Ej 1: Reglas de negocio complejas.
  - Ej 2: Dependencias lógicas entre procesos difíciles de simplificar.
  - Ej 3: Requisitos ambiguos que cambian según el contexto.
- **Accidental**: Surge por herramientas, procesos o malas decisiones. En general es reducible
  - Ej 1: Configuración complicada del entorno de desarrollo y despliegue.
  - Ej 2: Herramientas mal documentadas y poco estándar.
  - Ej 3: Malas decisiones de arquitectura.
  - Ej 4: Frameworks con curvas de aprendizaje innecesariamente elevadas.

Ciclo de Vida de Desarrollo de Software

- Planificación: Definición de objetivos, estimaciones de recursos, costes y cronogramas. Permite anticipar riesgos y organizar el proyecto
- Análisis: especificación de requisitos
- Diseño: Especificación en una arquitectura de sistema
- Implementación: Codificación
- Testeo: Asegurar que los requisitos se cumplan
- Deployment: Puesta en marcha en entorno productivo
- Mantenimiento: Corregir errores, mejorar implementación y adaptar el sistema a nuevas necesidades.



Principios de Diseño

KISS: Keep it Simple

SOLID:

- Single responsibility principle
- Open for extension, but closed for modification.
- Subtypes must be replaceable by their base types without issues.
- Clients should not be forced to depend upon interfaces that they do not use
- Depend upon abstractions

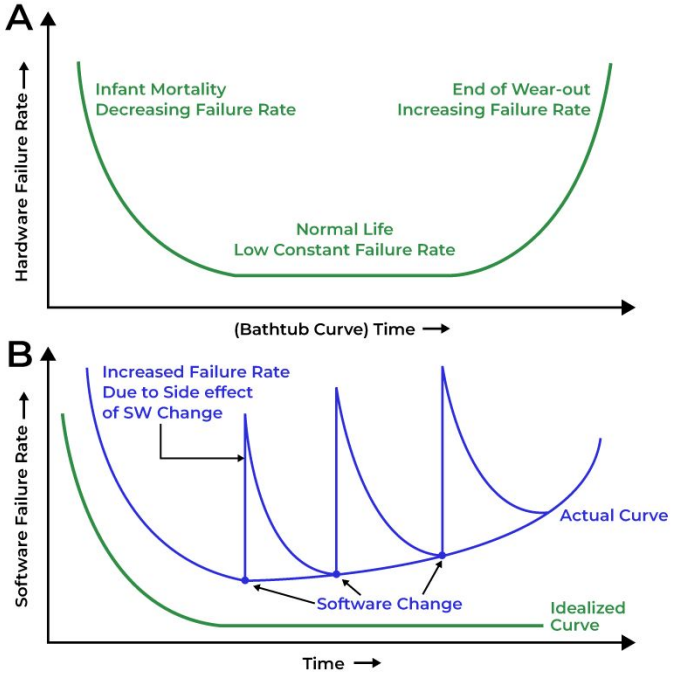
YAGNI: You aren't gonna need it

DRY: Don't repeat yourself

Errores y Deuda Técnica

Tasas de Error en Software

- Hardware: presenta la “curva de la bañera”: fallas iniciales, etapa estable y desgaste final.
- Software:
  - **Errores iniciales**: fallas provienen de defectos de programación no detectados en pruebas tempranas.
  - **Operación normal**: si bien no se desgasta cada nueva versión introduce errores iniciales, elevando nuevamente la tasa de fallas.
  - **Degradación**: cambios constantes acumulan defectos y pueden justificar reemplazar el software completamente.

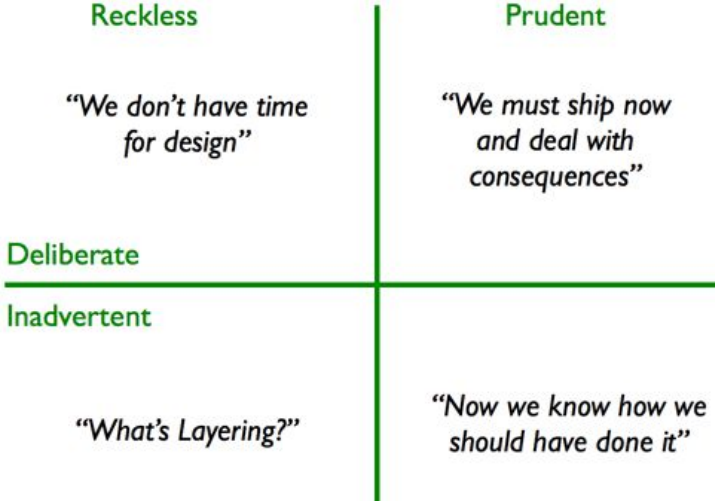


Deuda Técnica (1)

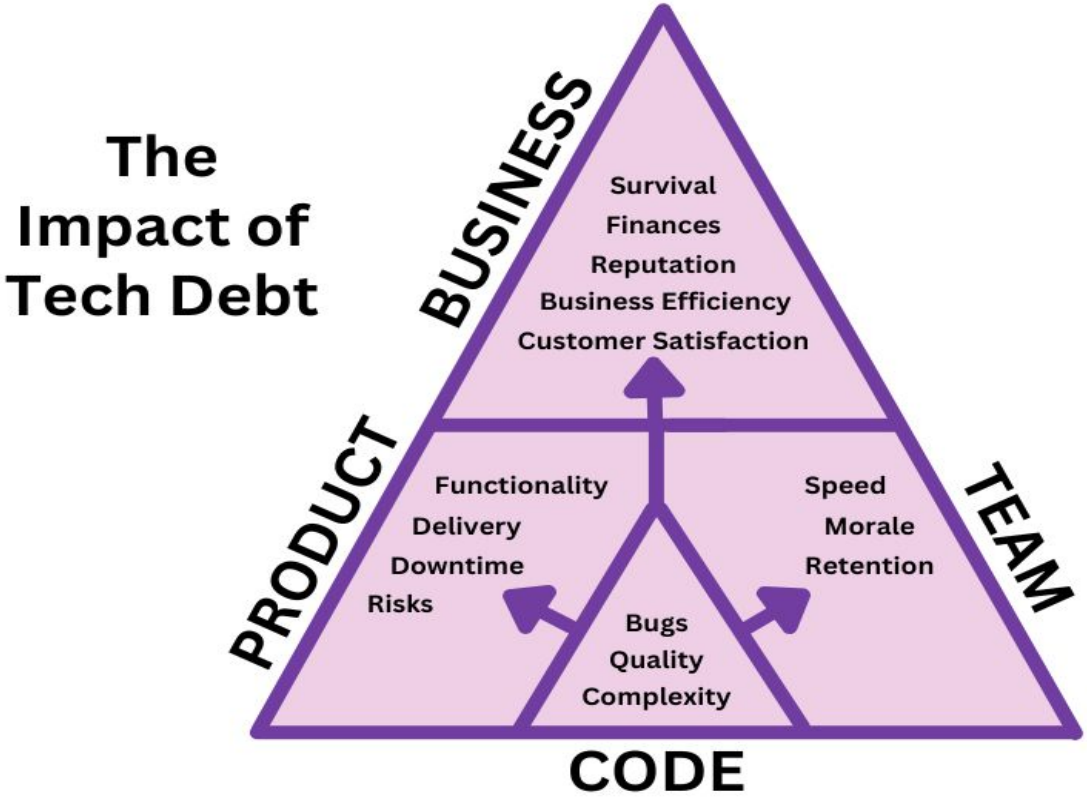


Deuda Técnica (2)

- Es un **atajo para ahorrar tiempo que genera "intereses"**: costos de mantenimiento, bugs, dificultad de cambio.
- **Causas**: presión, malas prácticas, desconocimiento técnico, decisiones apresuradas.
- **No toda deuda es mala**; lo importante es gestionar y pagarla antes de que bloquee la evolución del sistema.

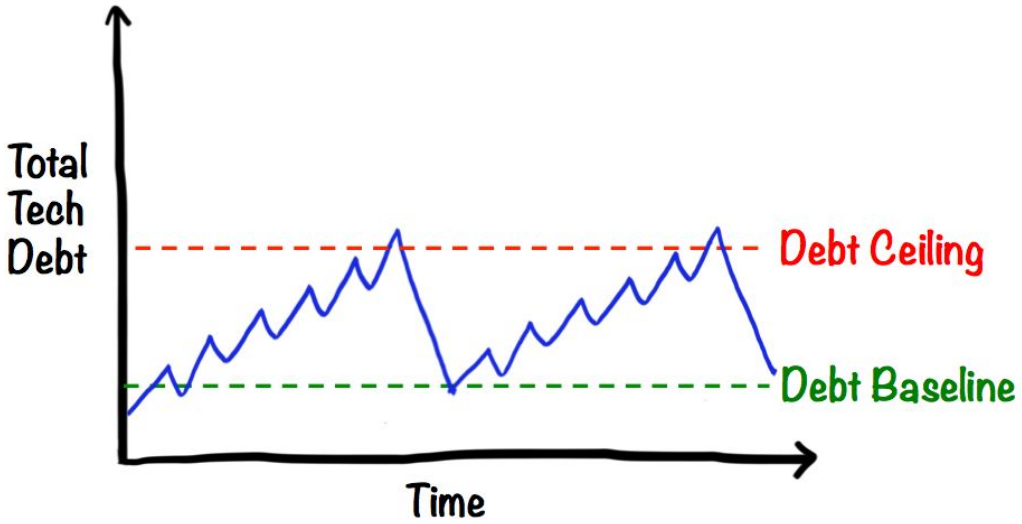


Deuda Técnica (3)



Refactorización

- Mejorar estructura interna sin alterar funcionalidad
- Facilita mantenibilidad y comprensión del sistema
- Reduce riesgos acumulados por deuda técnica



Casos de Falla

- Cohete Ariane 5 (1996)
  - Reutilización de software de Ariane 4 sin pruebas adecuadas → conversión numérica provocó overflow y explosión del cohete.

- Therac-25 (1985-1987)
  - Fallos en un sistema médico de radioterapia → dosis masivas de radiación, varios pacientes muertos o heridos.

- Y muchos más...

Dudas?

Clase 1 - Fundamentos de la ingeniería de software

Repaso de Git

Repaso de Git

-

Métricas de Productividad

Métricas tradicionales (y sus problemas):

- Líneas de código, número de commits → pueden ser engañosas.

Métricas modernas:

- DORA
    - Deployment Frequency
      Lead Time for Changes
      Change Failure Rate
      Mean Time to Restore
- Mean Time to Recovery (MTTR).
- Tasa de defectos.
- Cobertura de tests.

Concepto clave: No se trata de producir más código, sino de entregar valor.